APPENDIX B

Error Catalogue

This appendix lists every lifetime-related diagnostic that appears in the book: errors with a code, errors without one, lints, and the two run-time panics of `RefCell`. For each one it gives what the diagnostic usually means, the typical fixes, and where the book explains it, with the file that reproduces it. Every file named here is in `code/compile_fail/` (or `code/examples/` for the panics), and its `.stderr` snapshot shows the full diagnostic as rustc 1.99 prints it.

Two habits make any of these diagnostics easier to read (Chapter 1):

- Read the labels as a timeline: where a borrow is created, what conflicts with it or ends its value, which later use keeps it alive, and where a requirement comes from ("argument requires that ...", "returning this value requires that ...", "type annotation requires that ...").
- Treat a suggestion as one way to make *this* error disappear, not as the contract you want. Several suggestions in this book (noted below) compile and are still the wrong design.

`rustc --explain E0597` prints the long explanation of any code.

B.1 Errors with a code

E0106 – missing lifetime specifier

Means: a reference (or a type with a lifetime parameter) needs a lifetime that elision cannot provide.

- In a return type: more than one parameter has lifetimes and there is no `self` reference, or the only parameter has two lifetimes (`&[&str] -> &str`). Name the lifetime the result borrows from: `fn f<'a>(x: &'a str, y: &str) -> &'a str`. Writing `'_` in the output does not help. (Chapter 3; Chapter 6 for `'_`; `ch03_two_inputs`, `ch03_named_and_elided`, `ch03_rule2_shared_name`, `ch03_rule2_slice_of_refs`, `ch03_self_by_value`, `ch03_self_two_refs`, `ch06_ambiguous`)
- In a struct or enum field: fields are never elided. Declare the parameter, `struct S<'a> { r: &'a str }`, or own the data. (Chapters 2 and 3; `ch02_missing_lifetime`, `ch03_struct_field_elided`)
- In an `Fn(..)` bound, a function pointer type or a trait method: the same rules apply as for functions. In an `Fn` bound, the suggestion ties every lifetime together, which may be the wrong contract. (Chapters 3 and 9; `ch03_fn_pointer_two_inputs`, `ch03_trait_no_body`, `ch09_sugar_two_inputs`)

E0227 – ambiguous lifetime bound, explicit lifetime bound required

Means: a trait object's default lifetime bound would come from the trait, but the trait has two different non-`'static` lifetime bounds. Write the bound (`dyn Trait + 'a`). (Chapter 8; `ch08_two_trait_bounds`)

E0228 – cannot deduce the lifetime bound for this trait object type from context

Means: the type that contains a trait object has more than one lifetime bound on that parameter, so there is no default. Write the bound. (Chapter 8; `ch08_two_bounds`)

E0261 – use of undeclared lifetime name

Means: a lifetime such as `'a` is used without being declared in `<...>` on the function, `impl`, struct or trait. A `'_` in an `impl` header cannot be referred to by a name later. Declare it: `impl<'src> Parser<'src>`. (Chapters 3 and 6; `ch03_struct_field_placeholders`, `ch06_impl_header_name`)

E0277 – trait bound not satisfied

Two forms in this book:

- "`L` is not an iterator": a `for` loop over a lending iterator, which does not implement `Iterator`. Use `while let Some(item) = it.next()`. (Chapter 10; `ch10_for_loop`)
- "`{async block@...}` cannot be unpinned": an `Unpin` bound applied to a future that may be self-referential. Pin it first with `pin!` or `Box::pin`. (Chapter 11; `ch11_not_unpin`)

E0308 – mismatched types: "one type is more general than the other"

Means: a type with a fixed lifetime is used where a higher-ranked type is required: a `fn(&'static str)` where `for<'a> fn(&'a str)` is expected, or a future type parameter `Fut` for an async callback that borrows its argument. Use the higher-ranked type; for async callbacks use `AsyncFn`. (Chapter 9; `ch09_hr_subtyping`, `ch09_async_fut_param`)

E0308 also reports that coercions do not pass through containers: a `Vec<&String>` is not a `Vec<&str>`. (Chapter 4; `ch04_no_coercion_in_vec`)

E0309 – the parameter type `T` may not live long enough

Means: a bound or type requires `T: 'a` and nothing guarantees it. Add the bound (`T: 'a`), or avoid the requirement: in edition 2024, an `impl Trait` return type that captures `T` does not need `+ 'a`. (Chapters 8 and 9; `ch08_outlives_type_param`)

E0310 – the parameter type `T` may not live long enough (`'static`)

Means: generic code passes `T` to an API that requires `'static` (`thread::spawn`, `Box<dyn Any>`, a `'static` trait object). Add the bound to your signature (`T: Send + 'static`), so that callers are checked against it. (Chapter 7; `ch07_generic_spawn`)

E0316 – nested quantification of lifetimes

Means: a bound has two `for<..>` binders, as in `for<'a> F: for<'b> Fn(&'a str, &'b str)`. Use one binder with both lifetimes. (Chapter 9; `ch09_binder_limits`)

E0373 – closure may outlive the current function, but it borrows `x`

Means: a closure that borrows a local is passed to an API that requires `'static` (usually `thread::spawn`). Add `move` so that the closure owns its captures, or use `thread::scope`. The same error exists for async blocks. (Chapter 7; `ch07_spawn_borrow`, `ch07_static_table_no`; for an async block spawned on an executor, Chapter 11: `ch11_spawn_borrow_block`)

E0382 – borrow of moved value / use of moved value

Means, in this book: a `&mut T` (or an `Option<&mut T>`) was moved, because mutable references are not `Copy`, and then used again. Let the compiler reborrow (pass it directly to a function), or reborrow explicitly with `&mut *r` or `.as_deref_mut()`. (Chapter 5; `ch05_move_mut_ref`, `ch05_generic_moves`, `ch05_option_mut_moves`)

E0392 – lifetime parameter `'a` is never used

Means: a type declares a lifetime (or type) parameter that none of its fields uses, typically a wrapper around a raw pointer. Add a `PhantomData` field that says what the type behaves as if it held, such as `PhantomData<&'a T>`; the choice also sets the type's variance. (Chapter 13; `ch13_unused_lifetime`)

E0477 – the type does not fulfill the required lifetime

Means, in this book: an `impl` of a trait with a generic associated type does not repeat the trait's `where Self: 'a` on the associated type. Copy the where clause into the impl. (Chapter 10; `ch10_impl_without_where`, which also shows E0309 for the generic parameter)

E0499 – cannot borrow `x` as mutable more than once at a time

Means: two mutable borrows of the same place are live at the same point.

- Sequence the borrows so that the first is no longer used when the second starts. (Chapter 1; `ch01_two_mutable`)
- A method returns a reference with the elided `&mut self` lifetime, so every result keeps the value mutably borrowed. Return the data's lifetime instead (`-> Option<&'src str>`). (Chapter 2; `ch02_parser_elided`, `ch02_source_elided`)
- A method takes `&'a mut self` in `impl<'a> S<'a>`: "borrowed forever". Use the elided `&mut self`. (Chapters 2 and 4; `ch02_borrowed_forever`, `ch04_mut_self_lock`)
- Two methods on `&mut self` cannot split a value into disjoint parts, and indexing cannot prove two indices differ: borrow fields directly, or use `split_at_mut` / `get_disjoint_mut`. (Chapter 5; `ch05_split_methods`, `ch05_split_index`, `ch05_reborrow_frozen`)
- Two items of a lending iterator alive at once, or one item kept across loop iterations ("mutably borrowed here in the previous iteration of the loop"): each item borrows the iterator until it is dropped. (Chapter 10; `ch10_two_items`, `ch10_keep_line`)

E0502 – cannot borrow `x` as mutable because it is also borrowed as immutable (or the reverse)

Means: a shared and a mutable borrow of the same place are live at the same point.

- The classic: a reference into a collection is still used after a call that mutates it. Move the use, or copy the value out. (Chapter 1; `ch01_nll_conflict`, `ch01_nll_derived`, `ch01_mutable_then_shared`, `ch01_branch_conflict`, `ch01_loop_conflict`)
- An argument that borrows the receiver, where two-phase borrows do not apply. (Chapter 5; `ch05_two_phase_explicit`, `ch05_two_phase_shared`)
- A temporary in a `match` scrutinee, or (in edition 2021) an `if let` scrutinee, keeps a borrow alive in the arms. Bind it with `let` first. (Chapters 5 and 12; `ch05_match_scrutinee`, `ch05_if_let_2021`)
- A conditionally returned borrow (NLL problem case #3): accepted by Polonius on nightly; on stable, restructure (look up first, then insert). (Chapter 5; `ch05_nll_case3`)
- "Borrowed forever" (`&'a mut T<'a>`): the immutable borrow and "mutable borrow later used here" are on the same expression. (Chapters 3 and 14; `ch03_rule_unify_push`, `ch14_borrowed_forever`, `ch14_collector_forever`)
- Tokens borrowed from a buffer that is reused; an `impl Trait` in a trait that captures `&self`. (Chapters 8, 12 and 14; `ch14_tokens_outlive_buffer`, `ch08_rpitit_captures`, `ch12_oncecell_take`, `ch02_shared_self_keep`)
- A future holds the borrows of its arguments until it completes; a raw-pointer iterator holds a borrow through `PhantomData`. (Chapters 11 and 13; `ch11_future_holds_borrow`, `ch13_raw_iter_borrow`)

E0505 – cannot move out of `x` because it is borrowed

Means: a value is moved while a borrow of it is still going to be used. Use the borrow before the move, or clone the value. (Chapter 5; `ch05_move_while_borrowed`)

A self-referential value cannot be moved while it borrows from itself: "move out of `doc` occurs here" and "borrow later used here" point at the same spot. Store indices or ranges instead of references. (Chapter 13; `ch13_self_ref_move`, `ch13_self_ref_new`)

E0506 – cannot assign to `x` because it is borrowed

Means: an assignment would drop the old value while a borrow of it is live. Assign after the last use of the borrow. (Chapter 1; `ch01_assign_borrowed`)

E0515 – cannot return reference to local variable / value referencing local variable / reference to temporary value

Means: a function returns a borrow of something it owns, which is dropped when it returns.

- Return owned data (`String`, `Vec`), or let the caller own the input. (Chapters 5 and 14; `ch05_return_local`, `ch14_error_borrows_local`, `ch08_error_escapes`)
- A compiler suggestion tied the result to a local collection. (Chapter 3; `ch03_slice_suggested`)
- `&String::new()` or `&Cell::new(0)` is not promoted to a `'static` constant. (Chapter 7; `ch07_not_promoted`)
- A reference into a `RefCell` would outlive its guard: return the guard, `Ref::map`, or take a closure. (Chapter 12; `ch12_return_ref_from_guard`)

- With one lifetime parameter for two inputs, the result is tied to the shorter-lived one: use two parameters. (Chapter 14; `ch14_one_lifetime`)
- A correctly bounded `unsafe` API refuses to let a slice outlive its buffer. (Chapter 13; `ch13_bounded_slice`)

E0521 – borrowed data escapes outside of function (or closure)

Means: data borrowed from a parameter flows into a place that must outlive the function, typically `thread::spawn` or an `impl dyn Trait` block with an implicit `'static`. `move` does not help when the captured value is itself a reference. Own the data (clone, `Arc`), use `thread::scope`, or write `impl dyn Trait + '_`. For closures: a callback that stores its argument in an outer collection. The same error appears when an `async move` block captures a reference parameter and is spawned, and with the "current limitation" note of Chapter 10. (Chapters 6, 7, 10, 11 and 14; `ch07_spawn_param`, `ch06_impl_dyn_static`, `ch14_callback_keeps_borrow`, `ch11_spawn_ref_param`, `ch10_for_each_static`)

E0582 – binding for associated type `Output` references lifetime `'a`, which does not appear in the trait input types

Means: a higher-ranked lifetime appears only in an output, as in `for<'a> Fn() -> &'a str`. Nothing constrains it; use `'static` or tie it to an input. (Chapter 9; `ch09_binder_limits`)

E0597 – `x` does not live long enough

Means: a borrow would have to stay live after its value is dropped. The labels tell you why it must stay live:

- "borrow later used here": a use after the value's scope ended. Use the result earlier, move the value to an outer scope, or own the result. (Chapter 1; `ch01_longest_dangling`, `ch01_signature_contract`)
- "argument requires that `x` is borrowed for `'a`", where `'a` is a lifetime parameter of the function: a caller-chosen lifetime used for a local; make the bound higher-ranked. (Chapter 9; `ch09_caller_picks`, `ch09_borrowed_iter_named`)
- "… borrowed for `'static`" from an argument, a type annotation or a coercion: a `'static` requirement (a `T: 'static` bound, `&'static`, a default `Box<dyn Trait>` bound). Pass owned data. (Chapters 1, 3, 7 and 8; `ch01_annotation_static`, `ch03_static_field_local`, `ch07_static_bound_borrow`, `ch07_callback_borrow`, `ch08_trait_bounds_explicit`)
- A contract that ties the result to a short-lived input: give the inputs separate lifetimes. (Chapter 3; `ch03_rule_first`, `ch03_rule_unify`, `ch03_contract_v2`)
- Invariance: the type behind `&mut` or `Cell` cannot shrink, so a short borrow must be stored as a long one. (Chapter 4; `ch04_mut_invariance`, `ch04_cell_invariance`)
- "borrow might be used here, when `x` is dropped and runs the `Drop` code": the drop check; declare the borrowed value first. (Chapter 5; `ch05_drop_check`)
- Edition 2021 tail-expression temporaries ("… when that temporary is dropped"); edition 2024 fixes them. (Chapters 5 and 12; `ch05_tail_expr_2021`, `ch12_tail_guard_2021`)
- Edition 2024 over-capturing in `impl Trait`, with the note "this call may capture more lifetimes than intended": use `use<..>`. (Chapter 8; `ch08_overcapture`, `ch08_async_captures`)

- "borrow later stored here": an `async fn` future captures every input, even one its output does not mention; a future returned or spawned while it borrows a local. (Chapter 11; `ch11_future_outlives_key`, `ch11_return_borrowing_future`, `ch11_spawn_borrow_call`)
- A higher-ranked bound over a GAT, with the note "due to a current limitation of the type system, this implies a `'static` lifetime". (Chapter 10; `ch10_hrtb_static`)
- An invariant `PhantomData` marker refuses a short-lived value, or a generic `Drop` impl keeps a borrow alive until the drop. (Chapter 13; `ch13_out_slot_variance`, `ch13_dropck_log`)

E0637 – `'_` cannot be used here

Means: `'_` appears where a lifetime must be declared, such as an associated type or a where-clause bound. Use a named lifetime; for an associated type that should borrow from `&self`, use a generic associated type. (Chapters 6 and 10; `ch06_assoc_type`, `ch06_where_clause`, `ch10_item_anonymous`)

E0658 – unstable feature

Five unstable features in this book produce E0658 on stable Rust 1.99: closure lifetime binders (`for<'a> |x: &'a str| ...`), binders over types (`for<T>`), elided lifetimes inside argument-position `impl Trait`, return type notation (`T::method(..): Send`) and `#[may_dangle]`. (Chapters 8, 9, 11 and 13; `ch09_binder_limits`, `ch08_apit_anonymous`, `ch11_rtn_unstable`, `ch13_may_dangle_unstable`)

E0700 – hidden type captures lifetime that does not appear in bounds

Means: an `impl Trait` return type's hidden type uses a lifetime that the opaque type does not capture. In edition 2021 this is the default for elided lifetimes (the old fix was `+ '_`); in edition 2024, it happens when a `use<..>` list leaves out a lifetime the hidden type needs. Add the lifetime to `use<..>`. (Chapter 8; `ch08_rpit_2021`, `ch08_use_nothing`)

E0716 – temporary value dropped while borrowed

Means: a reference to a temporary outlives the statement that created the temporary. Temporary lifetime extension applies only to certain `let` positions, not through method or function calls. Bind the value with `let` first. (Chapter 5; `ch05_temporary_dropped`; Chapter 14: `ch14_builder_temporary`, and `ch14_iterator_overcaptures` for the edition 2024 variant)

E0794 – cannot specify lifetime arguments explicitly if late bound lifetime parameters are present

Means: a turbofish names a lifetime that is chosen at each call. Remove it. (Chapter 9; `ch09_late_turbofish`)

B.2 Diagnostics without a code

lifetime may not live long enough

The most common lifetime diagnostic. It means that a region constraint between two lifetimes cannot be proven. The primary label says which:

- "returning this value requires that `'1` must outlive `'static`": a borrowed value is returned as a trait object with the default `'static` bound (`Box<dyn Fn ...>`, `Box<dyn Error>`). Add `+ '_`, or own the data. (Chapters 6 and 8; `ch06_dyn_static`, `ch08_error_box_static`)
- "function was supposed to return data with lifetime `'a` but it is returning data with lifetime `'b`": the body breaks the signature; the help suggests a bound `'b: 'a`. (Chapters 1, 2 and 6; `ch01_body_vs_signature`, `ch02_lending_promise`, `ch06_impl_unrelated`)
- "returning this value requires that `'1` must outlive `'2`" inside a closure: a stored closure returns a reference to its argument. Pass it to a function with an `Fn` bound, route it through an identity helper, or use a `fn`. (Chapters 9 and 12; `ch09_closure_return`, `ch12_thread_local_escape`)
- "coercion requires that `'src` must outlive `'static`": a borrowing type converted to `Box<dyn Any>`. The help ("add an explicit `'src` lifetime bound") moves the error to the callers. (Chapter 7; `ch07_any_borrowed`)
- Notes "requirement occurs because of a mutable reference to ..." and "mutable references are invariant over their type parameter": a variance problem. (Chapter 4; `ch04_invariant_witnesses`, `ch04_invariant_cells`, `ch04_struct_invariant`)
- Two `'_` assumed equal; a `+ 'a` used as if it were a capture list. (Chapters 6 and 8; `ch06_never_equal`, `ch08_outlives_two`)
- "method was supposed to return data with lifetime `'s` ..." in an `Iterator` impl whose items would borrow from `&mut self`: a lending iterator, which needs a GAT. (Chapter 10; `ch10_iterator_windows`)
- Notes "… makes the generic argument ... invariant": an invariant `PhantomData` marker. A `ScopedJoinHandle` returned out of `thread::scope`; here the help "consider adding 'move' keyword" does not fix anything. (Chapter 13; `ch13_invariant_marker`, `ch13_scope_escape`, `ch13_scope_escape_move`)

implementation of `Trait` is not general enough

Means: something implements a trait (often `Fn` or `FnOnce`) for one specific lifetime, and a higher-ranked bound requires it for every lifetime. The notes name both: "for any lifetime `'1`" is the requirement, "for some specific lifetime `'2`" is what exists. Annotate closure parameters, remove bounds that make a lifetime early-bound, or use an owning type for `for<'de>`-style bounds. (Chapter 9; `ch09_trait_param_borrow`, `ch09_not_general_closure`, `ch09_early_bound`)

missing required bound on `Item`

Means: a generic associated type is used with `&'a self` or `&'a mut self` in a trait method, and the compiler requires the declaration to say `where Self: 'a`. The help gives the exact clause to add. (Chapter 10; `ch10_missing_bound`)

captured variable cannot escape `FnMut` closure body

Means: an `FnMut` closure returns a future that borrows the closure's own captures. Use an async closure with an `AsyncFnMut` bound. (Chapter 11; `ch11_fnmut_future_capture`)

future cannot be sent between threads safely

Means: a future that must be `Send` holds a non-`Send` value (such as an `Rc`) across an `.await` (note "this value is used across an await"), or a trait's `async fn` gives generic code no way to know its future is `Send`. Use `Arc`, drop the value before the await, or declare the trait method as `fn ... -> impl Future<Output = T> + Send`. (Chapter 11; `ch11_spawn_not_send`, `ch11_trait_future_not_send`)

associated type `Iterator::Item` is declared without lifetime parameters

Means: an attempt to implement `Iterator` with items that borrow from the iterator itself (a lending iterator). Use a separate type that borrows the data, yield owned items, or use a GAT-based lending trait. (Chapters 2 and 10; `ch02_lending_iterator`)

`impl Trait` must mention all type parameters in scope in `use<...>` / must mention the `Self` type of the trait

Means: a `use<..>` list may leave out lifetimes but not type parameters, and in a trait it must list `Self`. (Chapter 8; `ch08_use_all_types`, `ch08_rpitit_use_self`)

due to a current limitation of the type system, this implies a `'static` lifetime

Means (a note): a higher-ranked bound over a generic associated type with `where Self: 'a` can only hold if the type is `'static`. Put the bound on the GAT in the trait, or use a helper trait. (Chapters 9 and 10)

B.3 Lints

Lint	Default	Fires on	Chapter
`mismatched_lifetime_syntaxes` (Rust 1.89)	warn	one lifetime written two ways: "hiding a lifetime that's elided elsewhere is confusing", "eliding a lifetime that's named elsewhere is confusing"	3, 6 (`ch06_hidden_return`)
`elided_lifetimes_in_paths` (in `rust_2018_idioms`)	allow	any hidden lifetime parameter: "hidden lifetime parameters in types are deprecated"	6 (`ch06_hidden_param`)
`async_fn_in_trait`	warn	`async fn` in a public trait: "auto trait bounds cannot be specified"	11 (`ch11_async_fn_in_trait_lint`)
`let_underscore_lock`	deny	`let _ = mutex.lock()` , which drops the guard at once: "non-binding let on a synchronization lock"	12 (`ch12_let_underscore_lock`)
`if_let_rescope` , `tail_expr_drop_order` , `impl_trait_overcaptures` (in `rust_2024_compatibility`)	allow	code whose meaning changes in edition 2024: drop order of temporaries, `impl Trait` captures	5, 8

B.4 Run-time panics

`RefCell` moves the borrow rules to run time. Breaking them panics:

Message	Means	Chapter
"RefCell already borrowed"	`borrow_mut` while a `Ref` or `RefMut` is alive	12 (`ch12_double_borrow`)
"RefCell already mutably borrowed"	`borrow` while a `RefMut` is alive	12 (`ch12_double_borrow`)

`try_borrow` and `try_borrow_mut` return an error instead of panicking.